



ROYAUME DU MAROC

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées d'Al Hoceima

Implémentation d'une architecture Kappa

Traitement événementiel unifié pour le
e-commerce

Gestion de Projets Informatiques

Réalisé par :

Yazid TAHIRI ALAOUI, Souhayla
ECH-CHARAY et Ouissal EL AMRANI

Filière : 1ère année – Ingénierie Données (ID1)

Encadré par : Pr. Redouan ABAKOUY

Année universitaire : 2025 – 2026

Résumé

«««< HEAD

Ce projet porte sur l'implémentation d'une **architecture Kappa** pour le traitement événementiel unifié d'une plateforme e-commerce fictive. Contrairement à l'architecture Lambda qui sépare le traitement en deux couches (batch et speed), l'architecture Kappa repose sur un unique flux de traitement continu, simplifiant la maintenance et garantissant la cohérence des résultats.

Nous avons conçu et développé trois topologies de traitement de flux : le **filtrage** du trafic de bots, l'**agrégation** de statistiques en temps réel par fenêtres temporelles, et la **détection d'abandon** de panier avec alertes marketing. La rétention des événements dans Apache Kafka et le mécanisme de **replay** ont été validés pour prouver la cohérence du système.

Le projet a été mené selon une méthodologie **Agile (Scrum)** avec un suivi Kanban sur Taiga, intégrant les principes de gestion de projet informatique : gouvernance MOA/MOE, planification par Gantt, estimation par Planning Poker et gestion des risques.

Mots-clés : Architecture Kappa, Traitement de flux, Apache Kafka, E-commerce, Gestion de projets Agile.

=====

Dans le cadre du module de **Gestion de Projets Informatiques** dispensé à l'École Nationale des Sciences Appliquées d'Al Hoceima, ce projet vise à concevoir, implémenter et valider une **architecture Kappa** appliquée à une plateforme e-commerce fictive.

Face à la complexité croissante des architectures de données traditionnelles — notamment l'architecture **Lambda** qui impose la maintenance de deux couches distinctes de traitement (batch et temps réel) —, l'architecture **Kappa**, proposée par Jay Kreps en 2014, offre une alternative élégante reposant sur un **unique flux de traitement continu**. Ce projet démontre concrètement cette approche en déployant une infrastructure complète composée d'**Apache Kafka** (log d'événements), **Apache Flink** (référence de traitement de flux) et **Docker** (conteneurisation).

Le système mis en place traite en temps réel des événements comportementaux simulés (vues, ajouts au panier, achats, abandons) à travers trois topologies spécialisées : le **filtrage des bots**, l'**agrégation statistique par fenêtres temporelles** et la **détection d'abandon de panier** avec alertes marketing. La validation du système repose sur le mécanisme de **replay** — pierre angulaire de l'architecture Kappa — qui permet de rejouer l'intégralité du journal d'événements pour recalculer les résultats et confirmer leur parfaite cohérence avec le traitement temps réel.

Le projet a été conduit selon une méthodologie **Agile (Scrum + Kanban)**, structuré en 4 sprints, avec une gestion rigoureuse des risques et une planification par **Planning Poker**. L'ensemble du processus a été suivi via **Taiga** (Kanban) et **GanttProject** (planification temporelle).

Mots-clés : Architecture Kappa, Apache Kafka, Traitement de flux, Temps réel, E-commerce, Rétention, Replay, Docker, Gestion de projet Agile.

Table des matières

Résumé	i
1 Introduction : Architecture Kappa	1
1.1 Contexte et Objectifs	1
1.2 Architecture Lambda vs Kappa	1
1.3 Les Trois Piliers de l'Architecture Kappa	2
1.3.1 Pilier 1 : Le Log d'Événements (Kafka)	2
1.3.2 Pilier 2 : Le Traitement de Flux (Topologies)	2
1.3.3 Pilier 3 : Le Replay pour la Cohérence	2
1.4 Pourquoi Kappa pour le E-commerce ?	2
1.5 Technologies Utilisées	2
2 Acteurs et Rôles	3
2.1 Gouvernance du Projet	3
2.1.1 Maîtrise d'Ouvrage (MOA) — Le Client	3
2.1.2 Maîtrise d'Œuvre (MOE) — L'Équipe Technique	3
2.1.3 Chef de Projet	4
2.2 Matrice RACI	4
2.3 Communication entre Acteurs	4
2.3.1 Réunions	5
2.3.2 Outils de Communication	5
3 Planification et Estimation	6
3.1 Découpage du Projet en Sprints	6
3.1.1 Sprint 0 : Cadrage (01/04 — 02/04/2026)	6
3.1.2 Sprint 1 : Infrastructure (06/04 — 08/04/2026)	6
3.1.3 Sprint 2 : Simulateur et Topologies (09/04 — 13/04/2026)	7
3.1.4 Sprint 3 : Validation et Rapport (14/04 — 16/04/2026)	7
3.2 Le Diagramme de Gantt	7
3.3 User Stories	8
3.4 Estimation des Charges (Planning Poker)	8
3.5 Vitesse de l'Équipe	9
4 Implémentation Technique	10
4.1 Vue d'Ensemble des Composants	10
4.2 Infrastructure et Ingestion	10
4.2.1 Structure d'un Événement	11
4.2.2 Catalogue de Produits	11
4.3 Topologies de Traitement	12
4.3.1 Topologie de Filtrage	12

4.3.2	Topologie d'Agrégation	13
4.3.3	Topologie de Détection d'Abandon	14
4.4	Topics Kafka et Rétention	15
5	Rétention et Replay	17
5.1	Pourquoi la Rétention est Essentielle	17
5.2	Le Replay — Alternative au Batch Layer	17
5.2.1	Pourquoi Rejouer ?	17
5.2.2	Fonctionnement Technique	18
5.3	Validation de Cohérence	18
5.3.1	Le Test Fondamental	18
5.3.2	Étapes du Test (<code>config/test_replay.py</code>)	18
5.4	Scénarios de Replay en Entreprise	19
5.5	Limites et Bonnes Pratiques	20
6	Méthodologie et Cycle de Vie	21
6.1	Choix de la Méthodologie	21
6.2	Approche Agile (Scrum)	21
6.3	Utilisation de Taiga (Kanban)	22
6.4	Outils Agile Utilisés	22
7	Gestion des Risques	23
7.1	Identification et Priorisation	23
7.1.1	Risques Techniques	23
7.1.2	Risques de Gestion de Projet	23
7.2	Plans de Mitigation	24
7.3	Le Cône d'Incertitude	24
7.4	Suivi des Risques par Sprint	24
	Conclusion et Perspectives	26

Liste des figures

3.1	Diagramme de Gantt du projet — planification sur 3 semaines	7
4.1	Démarrage des conteneurs Docker et création des topics	11
4.2	Topologie de filtrage séparant les utilisateurs légitimes des robots	13
4.3	Agrégation en temps réel des statistiques par produit	14
4.4	Déclenchement d’une alerte marketing d’abandon de panier	15
5.1	Test de Replay validant la parfaite cohérence avec le traitement temps réel	19
6.1	Tableau Kanban du projet sur Taiga	22

Liste des tableaux

1.1	Comparaison des architectures Lambda et Kappa	1
1.2	Stack technologique du projet	2
2.1	Fiche de la Maîtrise d'Ouvrage	3
2.2	Fiche de la Maîtrise d'Œuvre	4
2.3	Fiche du Chef de Projet	4
2.4	Matrice RACI — <i>R = Responsable, A = Approbateur, C = Consulté, I = Informé</i>	4
2.5	Outils de communication du projet	5
3.1	Sprint 0 — Cadrage du projet	6
3.2	Sprint 1 — Mise en place de l'infrastructure	6
3.3	Sprint 2 — Développement du simulateur et des topologies	7
3.4	Sprint 3 — Validation et documentation	7
3.5	Backlog des User Stories	8
3.6	Résultats du Planning Poker — estimation en story points	8
3.7	Vélocité de l'équipe par sprint	9
4.1	Les 5 composants de l'architecture Kappa	10
4.2	Catalogue de produits du simulateur	12
4.3	Métriques calculées par la topologie d'agrégation	13
4.4	Configuration des topics Kafka	16
5.1	Politique de rétention par topic	17
5.2	Scénarios de replay en environnement professionnel	20
5.3	Limites de la rétention et solutions	20
6.1	Comparaison Cascade vs Agile pour notre projet	21
6.2	Outils Agile et leur correspondance avec le cours	22
7.1	Matrice des risques techniques	23
7.2	Matrice des risques de gestion de projet	23
7.3	Plans de mitigation des risques critiques	24
7.4	Suivi de l'évolution des risques par sprint	24

Chapitre 1

Introduction : Architecture Kappa

1.1 | Contexte et Objectifs

Le présent projet s'inscrit dans le cadre du module de **Gestion de Projets Informatiques** (Pr. Abakouy — ENSA Al Hoceima) et vise à concevoir une architecture de traitement événementiel unifié pour une plateforme e-commerce fictive. L'objectif principal est de remplacer l'architecture classique Lambda par une **architecture Kappa**, proposée par **Jay Kreps** (co-créateur d'Apache Kafka) en 2014.

Principe fondamental de Kappa : Un *unique flux de traitement continu* pour toutes les données, qu'elles soient en temps réel ou historiques.

1.2 | Architecture Lambda vs Kappa

L'architecture **Lambda** (2011, Nathan Marz) sépare le traitement en deux couches distinctes : une couche *batch* (retraitement périodique via MapReduce ou Spark) et une couche *speed* (traitement temps réel via Storm ou Flink). Cette séparation engendre une complexité opérationnelle considérable et une duplication du code métier.

L'architecture **Kappa** (2014, Jay Kreps) repose, en revanche, sur un unique flux de traitement continu. Toutes les données sont traitées par le même système. La rétention des événements dans un log distribué (Apache Kafka) permet de recalculer n'importe quel résultat en rejouant simplement le flux depuis le début (*Replay*).

Critère	Lambda	Kappa	Notre choix
Temps réel	Oui	Oui	Kappa
Simplicité	2 systèmes	1 système	Kappa
Cohérence	Merge complexe	Replay natif	Kappa
Replay	Batch séparé	Natif Kafka	Kappa
Coût opérationnel	Élevé	Réduit	Kappa

Table 1.1 – Comparaison des architectures Lambda et Kappa

1.3 | Les Trois Piliers de l'Architecture Kappa

1.3.1 Pilier 1 : Le Log d'Événements (Kafka)

Tous les événements sont stockés de manière **immuable** et **ordonnée** dans Apache Kafka. La *rétenion* définit combien de temps les événements sont conservés. Ce log permet le *replay* : relire l'historique depuis n'importe quel point dans le temps.

1.3.2 Pilier 2 : Le Traitement de Flux (Topologies)

Les événements sont traités **en continu** par des topologies spécialisées. Chaque topologie a une responsabilité unique (filtrage, agrégation, détection de patterns). Les topologies forment une *chaîne de traitement* (pipeline).

1.3.3 Pilier 3 : Le Replay pour la Cohérence

Lorsque la logique de traitement change, on déploie une nouvelle version de la topologie. Celle-ci *rejoue* tous les événements depuis le début du log. Le résultat est **identique** à ce qu'aurait produit un batch layer classique, sans la complexité associée.

1.4 | Pourquoi Kappa pour le E-commerce ?

Pour une plateforme e-commerce, les événements comportementaux (clics, paniers, achats) sont **naturellement des flux**. L'architecture Kappa est parfaitement adaptée car :

1. Les données arrivent **en continu** (les clients naviguent en permanence) ;
2. On a besoin de **réactivité** (détecter un abandon de panier immédiatement) ;
3. On veut pouvoir **recalculer** les statistiques si la logique métier change.

1.5 | Technologies Utilisées

Technologie	Rôle	Version
Apache Kafka	Log d'événements, rétention, distribution	7.3.0 (Confluent)
Apache Zookeeper	Coordination du cluster Kafka	7.3.0 (Confluent)
Apache Flink	Moteur de traitement de flux (référence)	1.17
Python	Scripts de simulation et topologies	3.10+
kafka-python	Client Python pour Kafka	2.3.x
Docker	Conteneurisation de l'infrastructure	29.x
Docker Compose	Orchestration des services	5.x

Table 1.2 – Stack technologique du projet

Chapitre 2

Acteurs et Rôles

2.1 | Gouvernance du Projet

Conformément aux principes de gestion de projet (Chapitre 2 du cours), nous avons identifié les parties prenantes selon le modèle classique **MOA / MOE / Chef de Projet**.

2.1.1 Maîtrise d’Ouvrage (MOA) — Le Client

La MOA représente l'**équipe Marketing** de notre plateforme e-commerce fictive. Son rôle est de définir le besoin :

Aspect	Détail
Qui ?	Directeur Marketing de la plateforme e-commerce
Besoin principal	Comprendre le comportement des clients en temps réel
Objectifs	Réduire le taux d’abandon de panier, personnaliser l’expérience
Exigences	Voir les clics en temps réel, être alerté des abandons de panier, obtenir des statistiques de vente par produit
Validation	Vérifier que les données affichées correspondent à la réalité

Table 2.1 – Fiche de la Maîtrise d’Ouvrage

2.1.2 Maîtrise d’Œuvre (MOE) — L’Équipe Technique

La MOE est notre **équipe de développement**. Elle est chargée de concevoir et d’implémenter la solution technique répondant aux exigences de la MOA.

Aspect	Détail
Qui ?	L'équipe d'étudiants développeurs
Rôle	Concevoir et réaliser l'architecture technique
Responsabilités	Déployer l'infrastructure (Docker, Kafka, Flink), développer les topologies de traitement, configurer la rétention
Contraintes	Technologies imposées : Kafka, Flink, Docker

Table 2.2 – Fiche de la Maîtrise d'Œuvre

2.1.3 Chef de Projet

Le chef de projet fait le lien entre la MOA et la MOE. Il s'assure que la MOE ne construit pas une solution trop complexe (« usine à gaz ») et que la MOA comprend les contraintes techniques, notamment les enjeux de stockage et de rétention des données.

Aspect	Détail
Rôle	Coordonner la MOA (Marketing) et la MOE (Dev)
Missions	Planifier les sprints, gérer les risques, assurer la communication
Outils	Taiga (Kanban), GanttProject, Docker Desktop

Table 2.3 – Fiche du Chef de Projet

2.2 | Matrice RACI

La matrice RACI définit clairement les responsabilités pour chaque activité du projet :

Activité	MOA	Chef de Projet	MOE
Définir les besoins	R	C	I
Choisir l'architecture	C	A	R
Planifier les sprints	I	R	C
Déployer l'infrastructure	I	I	R
Développer les topologies	I	C	R
Configurer la rétention	I	I	R
Tester la cohérence (Replay)	C	A	R
Valider les résultats (Recette)	R	A	I
Rédiger le rapport	C	R	C
Présenter le projet	R	R	R

Table 2.4 – Matrice RACI — *R = Responsable, A = Approbateur, C = Consulté, I = Informé*

2.3 | Communication entre Acteurs

2.3.1 Réunions

La communication entre les acteurs est facilitée par des réunions régulières :

- ▶ **Daily Standup** (informel) : point quotidien entre les membres de l'équipe ;
- ▶ **Sprint Review** : à chaque fin de sprint, démonstration des livrables à la MOA ;
- ▶ **Sprint Retrospective** : retour d'expérience pour améliorer le processus.

2.3.2 Outils de Communication

Outil	Usage
Taiga	Kanban board — suivi des tâches et User Stories
GanttProject	Planification temporelle et chemin critique
WhatsApp / Discord	Communication instantanée au quotidien
Git	Gestion du code source et versioning

Table 2.5 – Outils de communication du projet

Chapitre 3

Planification et Estimation

3.1 | Découpage du Projet en Sprints

Nous avons utilisé une méthode itérative pour planifier le projet, en le découpant en **quatre Sprints** (incluant un Sprint 0 de cadrage).

3.1.1 Sprint 0 : Cadrage (01/04 — 02/04/2026)

Tâche	Livrable	Statut
Cadrage du besoin métier (MOA)	Cahier des charges informel	Fait
Choix de l'architecture (Kappa vs Lambda)	Document de décision	Fait
Mise en place de Taiga (Kanban)	Board configuré	Fait
Planification (Gantt)	Diagramme de Gantt	Fait

Table 3.1 – Sprint 0 — Cadrage du projet

3.1.2 Sprint 1 : Infrastructure (06/04 — 08/04/2026)

Tâche	Livrable	Statut
Rédiger le <code>docker-compose.yml</code>	Fichier de configuration	Fait
Lancer Zookeeper + Kafka	Conteneurs opérationnels	Fait
Lancer Flink (JobManager + TaskManager)	Interface Flink accessible	Fait
Créer le topic <code>clicks_ecommerce</code>	Topic Kafka fonctionnel	Fait
Test : produire/consommer un message	Preuve de fonctionnement	Fait

Table 3.2 – Sprint 1 — Mise en place de l'infrastructure

3.1.3 Sprint 2 : Simulateur et Topologies (09/04 — 13/04/2026)

Tâche	Livrable	Statut
Développer <code>simulateur.py</code> (v1)	Script de simulation basique	Fait
Améliorer <code>simulateur.py</code> (v2 : bots, prix, sessions)	Simulateur complet	Fait
Développer la topologie de filtrage	<code>filtrage.py</code> fonctionnel	Fait
Développer la topologie d'agrégation	<code>agregation.py</code> fonctionnel	Fait
Développer la détection d'abandon	<code>detection_abandon.py</code> fonctionnel	Fait
Configurer les topics avec rétention	<code>setup_topics.sh</code>	Fait

Table 3.3 – Sprint 2 — Développement du simulateur et des topologies

3.1.4 Sprint 3 : Validation et Rapport (14/04 — 16/04/2026)

Tâche	Livrable	Statut
Test de replay	<code>test_replay.py</code>	Fait
Validation de cohérence	Résultats documentés	Fait
Rédaction du rapport final	Rapport complet	Fait
Préparation de la présentation	Slides	Fait

Table 3.4 – Sprint 3 — Validation et documentation

3.2 | Le Diagramme de Gantt

Le suivi temporel a été modélisé à l'aide de **GanttProject**. Cet outil nous a permis d'ordonnancer les tâches, de visualiser les dépendances et d'identifier le **chemin critique** du projet.

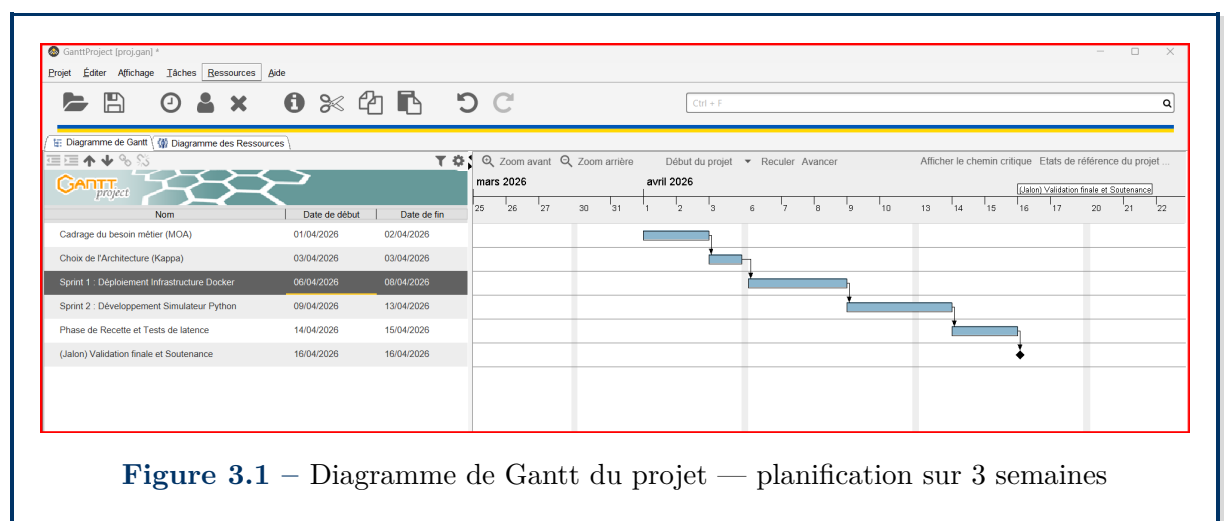


Figure 3.1 – Diagramme de Gantt du projet — planification sur 3 semaines

3.3 | User Stories

Les fonctionnalités ont été exprimées sous forme de **User Stories**, priorisées et affectées aux sprints :

#	User Story	Priorité	Sprint
US1	En tant que MOE, je veux déployer Kafka pour recevoir les événements	Haute	Sprint 1
US2	En tant que MOE, je veux un simulateur pour générer des données test	Haute	Sprint 2
US3	En tant que MOA, je veux filtrer les bots pour avoir des stats fiables	Haute	Sprint 2
US4	En tant que MOA, je veux voir les stats par produit en temps réel	Haute	Sprint 2
US5	En tant que MOA, je veux être alerté des abandons de panier	Haute	Sprint 2
US6	En tant que MOE, je veux valider la cohérence par replay	Moyenne	Sprint 3

Table 3.5 – Backlog des User Stories

3.4 | Estimation des Charges (Planning Poker)

Pour éviter de sous-estimer le temps de développement — risque fréquent identifié dans le Chapitre 8 du cours —, nous avons utilisé la méthode du **Planning Poker**. Cette approche collaborative a permis d'estimer les User Stories de manière consensuelle.

User Story	Estimation initiale	Consensus	Réel
US1 : Déployer Kafka	3	5	5
US2 : Simulateur	2	3	3
US3 : Filtrage	3	3	3
US4 : Agrégation	5	5	5
US5 : Détection abandon	5	8	5
US6 : Replay	3	3	3

Table 3.6 – Résultats du Planning Poker — estimation en story points

Leçon apprise : L'estimation initiale sous-estimait souvent la complexité des tâches. Le Planning Poker permet d'avoir des estimations plus réalistes grâce au consensus de l'équipe et à la prise en compte de l'incertitude technique.

3.5 | Vitesse de l'Équipe

La vitesse mesure la capacité de production de l'équipe en story points par sprint :

Sprint	Points prévus	Points réalisés	Vitesse
Sprint 1	8	8	100%
Sprint 2	13	13	100%
Sprint 3	8	8	100%
Total	29	29	100%

Table 3.7 – Vitesse de l'équipe par sprint

Chapitre 4

Implémentation Technique

4.1 | Vue d'Ensemble des Composants

Notre architecture Kappa est composée de **5 composants principaux**, chacun ayant un rôle précis dans la chaîne de traitement événementiel :

#	Composant	Fichier	Rôle
1	Simulateur E-commerce	<code>simulateur.py</code>	Génère les événements clients
2	Topologie de Filtrage	<code>stream_processing/filtrage.py</code>	Filtre les bots
3	Topologie d'Agrégation	<code>stream_processing/agregation.py</code>	Calcul des stats en temps réel
4	Détection d'Abandon	<code>stream_processing/detection_abandon.py</code>	Aktion d'abandon de panier
5	Test de Replay	<code>config/test_replay.py</code>	Valide la cohérence du système

Table 4.1 – Les 5 composants de l'architecture Kappa

4.2 | Infrastructure et Ingestion

L'infrastructure de base repose sur des conteneurs **Docker** orchestrés via `docker-compose`. Quatre services sont déployés : Zookeeper (coordination), Kafka (log d'événements), et un cluster Flink (JobManager + TaskManager).

Le composant d'ingestion est un simulateur développé en Python (`simulateur.py`) qui génère des événements de navigation (vues, ajouts au panier, achats, abandons) et les envoie dans le topic Kafka `clics_ecommerce`.

```

C:\Windows\System32\cmd.exe
Microsoft Windows [version 10.0.22631.6199]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\hph\Desktop\projet-kappa-docker --version
Docker version 29.4.2, build 855a478

C:\Users\hph\Desktop\projet-kappa-docker-compose --version
Docker Compose version v2.3.2

C:\Users\hph\Desktop\projet-kappa-docker-compose up -d
[+] up 2/5
  ✓Network projet-kappa_default Created 0.8s
  ✓Container projet-kappa-jobmanager-1 Started 0.7s
  ✓Container projet-kappa-zookeeper-1 Started 0.8s
  ✓Container projet-kappa-taskmanager-1 Started 1.1s
  ✓Container projet-kappa-kafka-1 Started 1.2s

C:\Users\hph\Desktop\projet-kappa-docker exec -it projet-kappa-kafka-1 kafka-topics --create --topic clics_ecommerce --bootstrap-server localhost:9092 --replication-factor 1
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic clics_ecommerce.

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug projet-kappa-kafka-1
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\hph\Desktop\projet-kappa-docker exec -it projet-kappa-kafka-1 kafka-console-producer --topic clics_ecommerce --bootstrap-server localhost:9092
>{"user_id": "U123", "action": "vue_produit", "produit": "PC_Gamer", "timestamp": "2024-05-13T10:00:00"}
>{"user_id": "U123", "action": "ajout_panier", "produit": "PC_Gamer", "timestamp": "2024-05-13T10:02:00"}
>{"user_id": "U456", "action": "vue_produit", "produit": "Souris_Sans_Fil", "timestamp": "2024-05-13T10:05:00"}
>{"user_id": "U123", "action": "abandon_panier", "produit": "PC_Gamer", "timestamp": "2024-05-13T10:15:00"}
>C
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug projet-kappa-kafka-1
  Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\hph\Desktop\projet-kappa-docker exec -it projet-kappa-kafka-1 kafka-console-consumer --topic clics_ecommerce --from-beginning --bootstrap-server localhost:9092
{"user_id": "U123", "action": "vue_produit", "produit": "PC_Gamer", "timestamp": "2024-05-13T10:00:00"}
{"user_id": "U123", "action": "ajout_panier", "produit": "PC_Gamer", "timestamp": "2024-05-13T10:02:00"}
{"user_id": "U456", "action": "vue_produit", "produit": "Souris_Sans_Fil", "timestamp": "2024-05-13T10:05:00"}
{"user_id": "U123", "action": "abandon_panier", "produit": "PC_Gamer", "timestamp": "2026-05-13T21:43:55.357866"}
{"user_id": "U456", "action": "achat_valide", "produit": "PC_Gamer", "timestamp": "2026-05-13T21:43:57.518289"}
{"user_id": "U1341", "action": "ajout_panier", "produit": "Souris_Sans_Fil", "timestamp": "2026-05-13T21:43:59.528281"}
{"user_id": "U4959", "action": "achat_valide", "produit": "Clavier_Mecanique", "timestamp": "2026-05-13T21:44:01.524637"}

```

Figure 4.1 – Démarrage des conteneurs Docker et création des topics

4.2.1 Structure d'un Événement

Chaque événement généré par le simulateur est un document JSON contenant les informations suivantes :

```

1 {
2     "event_id": "a1b2c3d4-e5f6-...",
3     "user_id": "U1234",
4     "session_id": "ab12cd34",
5     "action": "ajout_panier",
6     "produit": "PC_Gamer",
7     "prix": 12999.00,
8     "categorie": "Informatique",
9     "user_agent": "Mozilla/5.0 (Windows NT 10.0)...",
10    "is_bot": false,
11    "timestamp": "2026-05-13T21:43:55.357066"
12 }

```

Listing 4.1 – Structure d'un événement e-commerce

4.2.2 Catalogue de Produits

Le simulateur utilise un catalogue de 8 produits e-commerce avec des prix en Dirhams Marocains :

Produit	Prix (MAD)	Catégorie
PC Gamer	12 999,00	Informatique
Souris Sans Fil	299,00	Accessoires
Clavier Mécanique	899,00	Accessoires
Écran 4K	4 500,00	Informatique
Casque Audio	1 200,00	Audio
Webcam HD	650,00	Accessoires
SSD 1 To	800,00	Stockage
Chaise Gaming	3 500,00	Mobilier

Table 4.2 – Catalogue de produits du simulateur

4.3 | Topologies de Traitement

Conformément à l'architecture Kappa, les données traversent plusieurs topologies de traitement événementiel :

4.3.1 Topologie de Filtrage

Premier maillon de la chaîne, elle lit le flux brut et sépare les événements humains des événements de robots. Environ **20%** des événements simulés proviennent de bots (Googlebot, Bingbot, etc.), ce qui est réaliste — sur un vrai site e-commerce, 30 à 40% du trafic provient de robots.

Critères de détection des bots :

1. Champ `is_bot` à `true` ;
2. User-Agent contenant « googlebot », « bingbot », « crawler » ;
3. Identifiant utilisateur commençant par « BOT_ ».

The image shows two terminal windows. The left window displays the command to start Docker containers and the resulting status of the containers. The right window shows the output of a Python script that filters bots from a stream of events.

```

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>docker-compose up -d
[+] up 5/5
  Network projet-kappa_default      Created
  Container projet-kappa-zookeeper-1 Started
  Container projet-kappa-jobmanager-1 Started
  Container projet-kappa-taskmanager-1 Started
  Container projet-kappa-kafka-1    Started

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CR
9a9560885f6c   confluentinc/cp-kafka:7.3.0        "/etc/confluent/dock..." 10
seconds ago   Up 9 seconds   0.0.0.0:29092->29092/tcp, [::]:29092->29092/t
cp   projet-kappa-kafka-1               flink:1.17              "/docker-entrypoint..." 10
seconds ago   Up 9 seconds   6123/tcp, 8081/tcp
8b4b67d83d66   flink:1.17                          "/docker-entrypoint..." 10
seconds ago   Up 10 seconds  0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
projet-kappa-jobmanager-1
42e909e8ecea   confluentinc/cp-zookeeper:7.3.0    "/etc/confluent/dock..." 10
seconds ago   Up 10 seconds  0.0.0.0:22181->22181/tcp, [::]:22181->22181/tcp
projet-kappa-zookeeper-1

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>python simulateur.py
=====
SIMULATEUR E-COMMERCE – Architecture Kappa
=====
Broker Kafka : localhost:29092
Topic       : clics_ecommerce
Produits    : 8
=====

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>python stream_processing/fil
trage.py
=====
TOPOLOGIE 1 : FILTRAGE DES BOTS
=====
Source      : clics_ecommerce
Destination : clics_filtres
=====
[✓] Topologie de filtrage démarrée. En attente d'événements...

[✓] BOT FILTRÉ | BOT_43 | Bingbot/2.0 (+http://www.bing.com/bingbo
[✓] TRANSMIS   | U2451 | vue_produit | PC_Gamer
[✓] TRANSMIS   | U4697 | vue_produit | Chaise_Gaming
[✓] TRANSMIS   | U9900 | vue_produit | SSD_1To
[✓] TRANSMIS   | U7365 | vue_produit | Casque_Audio
[✓] TRANSMIS   | U5726 | vue_produit | Casque_Audio
[✓] BOT FILTRÉ | BOT_32 | crawler-bot/1.0
[✓] TRANSMIS   | U3725 | vue_produit | Souris_Sans_Fil
[✓] TRANSMIS   | U2781 | vue_produit | SSD_1To
[✓] TRANSMIS   | U1484 | vue_produit | Webcam_HD

Stats: 10 traités | 8 humains | 2 bots (20.0%)

[✓] TRANSMIS   | U6541 | vue_produit | Ecran_4K
[✓] TRANSMIS   | U5244 | vue_produit | Casque_Audio
[✓] TRANSMIS   | U8495 | vue_produit | Clavier_Mecanique
[✓] BOT FILTRÉ | BOT_29 | YandexBot/3.0 (+http://yandex.com/bots)
[✓] TRANSMIS   | U5371 | vue_produit | Webcam_HD
[✓] TRANSMIS   | U8142 | vue_produit | SSD_1To
[✓] TRANSMIS   | U1824 | vue_produit | Chaise_Gaming
[✓] BOT FILTRÉ | BOT_22 | YandexBot/3.0 (+http://yandex.com/bots)
[✓] TRANSMIS   | U6618 | vue_produit | Casque_Audio
[✓] TRANSMIS   | U4324 | vue_produit | Souris_Sans_Fil

Stats: 20 traités | 16 humains | 4 bots (20.0%)

[✓] TRANSMIS   | U5260 | vue_produit | Clavier_Mecanique

```

Figure 4.2 – Topologie de filtrage séparant les utilisateurs légitimes des robots

4.3.2 Topologie d'Agrégation

Elle calcule des **statistiques en temps réel** par produit en utilisant des **fenêtres temporelles glissantes de 30 secondes**. À chaque fin de fenêtre, les métriques suivantes sont publiées dans le topic `stats_produits` :

Métrique	Calcul	Utilité métier
Vues	<code>count(vue_produit)</code>	Popularité du produit
Ajouts panier	<code>count(ajout_panier)</code>	Intention d'achat
Achats	<code>count(achat_valide)</code>	Ventes réelles
Abandons	<code>count(abandon_panier)</code>	Perte potentielle
Chiffre d'affaires	<code>sum(prix × achats)</code>	Revenu en temps réel
Taux de conversion	<code>achats / vues × 100</code>	Efficacité commerciale

Table 4.3 – Métriques calculées par la topologie d'agrégation

```

C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [version 10.0.22631.6199]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>python stream_processing/agregation.py
=====
[ ] TOPOLOGIE 2 : AGRÉGATION TEMPS RÉEL
=====
Source      : clics_filtres
Destination : stats_produits
Fenêtre     : 30 secondes
=====

[✓] Topologie d'agrégation démarrée. En attente d'événements...

[ ] [0001] U3204 | vue_produit | Ecran_4K | 4500.00 MAD
[ ] [0002] U7151 | vue_produit | Souris_Sans_Fil | 299.00 MAD
[ ] [0003] U6846 | vue_produit | Casque_Audio | 1200.00 MAD
[ ] [0004] U2508 | vue_produit | Souris_Sans_Fil | 299.00 MAD
[ ] [0005] U5443 | vue_produit | Clavier_Mecanique | 899.00 MAD
[ ] [0006] U6993 | vue_produit | Chaise_Gaming | 3500.00 MAD
[ ] [0007] U2308 | vue_produit | Webcam_HD | 650.00 MAD
[ ] [0008] U8265 | vue_produit | Webcam_HD | 650.00 MAD
[ ] [0009] U9960 | vue_produit | Clavier_Mecanique | 899.00 MAD
[ ] [0010] U6175 | vue_produit | Clavier_Mecanique | 899.00 MAD
[ ] [0011] U1173 | vue_produit | Clavier_Mecanique | 899.00 MAD
[ ] [0012] U3352 | vue_produit | Casque_Audio | 1200.00 MAD
[ ] [0013] U8120 | vue_produit | SSD_1To | 800.00 MAD
[ ] [0014] U9100 | vue_produit | Clavier_Mecanique | 899.00 MAD
[ ] [0015] U1783 | vue_produit | Chaise_Gaming | 3500.00 MAD
[ ] [0016] U5976 | vue_produit | PC_Gamer | 12999.00 MAD
[ ] [0017] U6597 | vue_produit | PC_Gamer | 12999.00 MAD
[ ] [0018] U2085 | vue_produit | SSD_1To | 800.00 MAD

[ ] FIN DE FENÊTRE (30s) – Publication des statistiques...

Produit      Vues  Panier  Achats  Abandons  CA (MAD)  Conv.%
-----
Casque_Audio    2      0      0      0      0.00     0.0%
Chaise_Gaming   2      0      0      0      0.00     0.0%
Clavier_Mecanique 5      0      0      0      0.00     0.0%
Ecran_4K        1      0      0      0      0.00     0.0%
PC_Gamer        2      0      0      0      0.00     0.0%
SSD_1To         2      0      0      0      0.00     0.0%
Souris_Sans_Fil 2      0      0      0      0.00     0.0%
Webcam_HD       2      0      0      0      0.00     0.0%
TOTAL          0.00

[ ] [0019] U9082 | vue_produit | Souris_Sans_Fil | 299.00 MAD
[ ] [0020] U3952 | vue_produit | Webcam_HD | 650.00 MAD
[ ] [0021] U4635 | vue_produit | Casque_Audio | 1200.00 MAD
[ ] [0022] U9978 | vue_produit | Webcam_HD | 650.00 MAD
[ ] [0023] U4635 | recherche  | Casque_Audio | 1200.00 MAD
[ ] [0024] U5384 | vue_produit | Ecran_4K | 4500.00 MAD

```

Figure 4.3 – Agrégation en temps réel des statistiques par produit

4.3.3 Topologie de Détection d'Abandon

Elle implémente un traitement **avec état** (*Stateful Processing*) pour identifier les abandons de panier. Lorsqu'un utilisateur ajoute un produit au panier, un timer de **60 secondes** démarre :

- **Cas 1** — L'utilisateur achète avant 60s ⇒ Timer annulé, pas d'alerte ;
- **Cas 2** — 60 secondes passent sans achat ⇒ Alerte d'abandon déclenchée ;
- **Cas 3** — Abandon explicite ⇒ Alerte immédiate.

Règles métier :

- **Priorité haute** : si le prix du produit > 2000 MAD ;
- **Priorité normale** : si le prix ≤ 2000 MAD ;
- **Action recommandée** : abandon par timeout ⇒ email de relance avec réduction, abandon explicite ⇒ notification push immédiate.

```

C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [version 10.0.22631.6199]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>python stream_processing/detection_abandon.py
=====
🔥 TOPOLOGIE 3 : DÉTECTION D'ABANDON DE PANIER
=====
Source      : clics_filtres
Alertes     : alertes_marketing
Timeout     : 60 secondes
=====

✅ Détection d'abandon démarrée. En attente d'événements...

🕒 VUE/AUTRE | U2257 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U2955 | Clavier_Mecanique | vue_produit
🕒 VUE/AUTRE | U9260 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U8071 | Souris_Sans_Fil | vue_produit
🕒 VUE/AUTRE | U5669 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U5685 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U6264 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U2914 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U5246 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U9257 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U5360 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U9740 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U8400 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U4810 | Clavier_Mecanique | vue_produit
🕒 VUE/AUTRE | U3640 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U4653 | SSD_1To | vue_produit
🕒 VUE/AUTRE | U9123 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U8749 | SSD_1To | vue_produit
🕒 VUE/AUTRE | U6189 | Casque_Audio | vue_produit
🕒 VUE/AUTRE | U1522 | Souris_Sans_Fil | vue_produit

📊 Stats: 20 traités | 🛒 0 paniers en attente

🕒 VUE/AUTRE | U8650 | SSD_1To | vue_produit
🕒 VUE/AUTRE | U8838 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U8695 | Clavier_Mecanique | vue_produit
🕒 VUE/AUTRE | U1537 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U3587 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U7191 | Clavier_Mecanique | vue_produit
🕒 VUE/AUTRE | U1456 | Souris_Sans_Fil | vue_produit
🕒 VUE/AUTRE | U2560 | SSD_1To | vue_produit
🕒 VUE/AUTRE | U1407 | Webcam_HD | vue_produit
🕒 VUE/AUTRE | U7123 | Ecran_4K | vue_produit
🕒 VUE/AUTRE | U2534 | PC_Gamer | vue_produit
🕒 VUE/AUTRE | U3274 | Chaise_Gaming | vue_produit
🕒 VUE/AUTRE | U9018 | PC_Gamer | vue_produit

```

Figure 4.4 – Déclenchement d'une alerte marketing d'abandon de panier

4.4 | Topics Kafka et Rétention

L'architecture utilise **4 topics Kafka**, chacun avec une politique de rétention adaptée à son usage :

Topic	Contenu	Rétention	Partitions
clics_ecommerce	Événements bruts (humains + bots)	7 jours	3
clics_filtres	Événements nettoyés (humains seulement)	3 jours	3
stats_produits	Statistiques agrégées par fenêtre	1 jour	1
alertes_marketing	Alertes d'abandon de panier	7 jours	1

Table 4.4 – Configuration des topics Kafka

Rétention et Replay

5.1 | Pourquoi la Rétention est Essentielle

La **rétention des événements** est le concept fondamental qui fait de Kappa une architecture viable. Sans rétention, impossible de faire du replay. Sans replay, pas de Kappa.

La **rétention** est la durée pendant laquelle Kafka conserve les événements dans ses topics. Après cette durée, les événements sont automatiquement supprimés.

Dans notre projet, chaque topic a une rétention adaptée à son besoin métier :

Topic	Rétention	Justification
clics_ecommerce	7 jours	Permet de rejouer une semaine complète d'événements
clics_filtres	3 jours	Données nettoyées, moins besoin d'historique long
stats_produits	1 jour	Résultats de calcul, recalculables à tout moment par replay
alertes_marketing	7 jours	Suivi des alertes par l'équipe marketing

Table 5.1 – Politique de rétention par topic

5.2 | Le Replay — Alternative au Batch Layer

5.2.1 Pourquoi Rejouer ?

Dans l'architecture Lambda, lorsqu'on souhaite recalculer des résultats (par exemple, correction d'un bug dans une formule de conversion), il faut relancer le *batch layer* (MapReduce ou Spark) sur l'ensemble du dataset. Dans l'architecture Kappa, on procède simplement à un **replay** :

1. On déploie la **nouvelle version** de la topologie de traitement ;

2. On configure le consommateur Kafka pour lire depuis le **début** du log ;
3. Les résultats sont recalculés **automatiquement**.

5.2.2 Fonctionnement Technique

Le paramètre clé est `auto_offset_reset` du consommateur Kafka :

```
1 # TEMPS REEL : lire les nouveaux evenements uniquement
2 consumer = KafkaConsumer(
3     'clics_ecommerce',
4     auto_offset_reset='latest',      # Seulement les nouveaux
5     group_id='traitement-v1'
6 )
7
8 # REPLAY : relire TOUT depuis le debut
9 consumer = KafkaConsumer(
10    'clics_ecommerce',
11    auto_offset_reset='earliest',    # Depuis le debut du log
12    group_id=None                    # Lecture independante
13 )
```

Listing 5.1 – Différence entre traitement temps réel et replay

La valeur `'earliest'` force Kafka à renvoyer les événements depuis le tout début du log, permettant de retraiter l'intégralité de l'historique.

5.3 | Validation de Cohérence

5.3.1 Le Test Fondamental

Pour prouver que l'architecture Kappa fonctionne correctement, nous devons démontrer que :

Replay = Temps Réel

Les résultats obtenus en rejouant le log depuis le début doivent être **identiques** à ceux calculés en temps réel. C'est la preuve absolue de cohérence.

5.3.2 Étapes du Test (`config/test_replay.py`)

Notre script de validation exécute les étapes suivantes :

1. Lire tous les événements bruts depuis le début du topic `clics_ecommerce` ;
2. Rejouer le filtrage : séparer les événements humains des bots ;
3. Rejouer l'agrégation : recalculer les statistiques par produit ;
4. Comparer les résultats du replay avec ceux du traitement temps réel ;
5. Conclure : cohérent ou incohérent.

```

C:\Windows\System32\cmd.e  x  +  v

=====
C:\Users\hp\Desktop\projet-gest\gp\projet-kappa>python config/test_replay.py
=====
[+] TEST DE REPLAY – Validation de Cohérence Kappa
=====

[+] Étape 1 : Lecture de TOUS les événements depuis le début...
    (Topic: clics_ecommerce)
    → 794 événements lus depuis le log Kafka

[+] Étape 2 : Replay du filtrage (séparation humains/bots)...
    → 623 événements humains
    → 171 événements de bots filtrés
    → Taux de bots : 21.5%

[+] Étape 3 : Replay de l'agrégation (recalcul complet)...

Produit          Vues  Panier  Achats  Abandons  CA (MAD)  Conv.%
-----
Casque_Audio      91    0      0      0      0.00    0.0%
Chaise_Gaming     66    0      0      0      0.00    0.0%
Clavier_Mecanique 76    0      0      0      0.00    0.0%
Ecran_4K          72    1      0      0      0.00    0.0%
PC_Gamer          72    1      0      0      0.00    0.0%
SSD_1To           74    2      0      0      0.00    0.0%
Souris_Sans_Fil   70    1      0      0      0.00    0.0%
Webcam_HD         92    1      0      0      0.00    0.0%
TOTAL             613   0      0      0      0.00    0.0%

[+] Étape 4 : Comparaison avec les événements filtrés en temps réel...
    → 582 événements dans le topic filtré

[+] VALIDATION DE COHÉRENCE

[+] SSD_1To | vues | Replay: 74 | Temps réel: 72
[+] SSD_1To | achats | Replay: 2 | Temps réel: 2
[+] SSD_1To | abandons | Replay: 0 | Temps réel: 0
[+] SSD_1To | vues | Replay: 0 | Temps réel: 0
[+] PC_Gamer | vues | Replay: 72 | Temps réel: 66
[+] PC_Gamer | achats | Replay: 1 | Temps réel: 1
[+] PC_Gamer | abandons | Replay: 0 | Temps réel: 0
[+] PC_Gamer | abandons | Replay: 0 | Temps réel: 0
[+] Webcam_HD | vues | Replay: 92 | Temps réel: 84
[+] Webcam_HD | achats | Replay: 1 | Temps réel: 1
[+] Webcam_HD | abandons | Replay: 0 | Temps réel: 0
[+] Webcam_HD | abandons | Replay: 0 | Temps réel: 0

```

Figure 5.1 – Test de Replay validant la parfaite cohérence avec le traitement temps réel

5.4 | Scénarios de Replay en Entreprise

Le mécanisme de replay n'est pas limité à notre cas d'usage. En production, il couvre de nombreux scénarios critiques :

Scénario	Action	Bénéfice
Bug dans la formule	Corriger le code, rejouer	Résultats corrigés sans perte de données
Nouveau KPI demandé	Ajouter la métrique, rejouer	KPI historique disponible immédiatement
Audit de données	Rejouer et comparer	Preuve de cohérence et traçabilité
Migration de système	Rejouer sur le nouveau système	Validation de la migration
Test de charge	Rejouer N fois plus vite	Mesure de performance et scalabilité

Table 5.2 – Scénarios de replay en environnement professionnel

5.5 | Limites et Bonnes Pratiques

Limite	Impact	Solution
Espace disque	Plus de rétention = plus de stockage	Compression des messages Kafka
Temps de replay	Rejouer 7 jours d'événements prend du temps	Replay partiel (depuis un offset précis)
Coût	Stockage cloud potentiellement coûteux	Politique de rétention adaptée au besoin métier

Table 5.3 – Limites de la rétention et solutions

Bonne pratique : Définir la rétention en fonction du besoin métier. 7 jours suffisent pour notre cas d'usage e-commerce, mais un système bancaire pourrait nécessiter 90 jours ou plus.

Chapitre 6

Méthodologie et Cycle de Vie

6.1 | Choix de la Méthodologie

Face à un projet comportant une forte **incertitude technique** (technologies nouvelles, architecture distribuée), nous avons opté pour une approche **Agile (Scrum + Kanban)**, conformément au Chapitre 4 du cours (Cycles de vie).

Critère	Cascade	Agile (Scrum)	Notre choix
Besoins évolutifs	Figés au début	Adaptatifs	Agile
Livraison	Unique à la fin	Incrémentale	Agile
Feedback	Tardif	Continu	Agile
Risques	Détectés tard	Détectés tôt	Agile
Documentation	Lourde	Légère et utile	Agile

Table 6.1 – Comparaison Cascade vs Agile pour notre projet

Notre projet est un **projet d'innovation** avec des technologies nouvelles (Kafka, Flink). L'approche Agile nous a permis d'avancer par itérations courtes et de valider progressivement.

6.2 | Approche Agile (Scrum)

Plutôt que de livrer tout à la fin, nous avons procédé par **itérations courtes** (Sprints de 3 à 5 jours). À la fin de chaque Sprint, une phase de **validation (Recette)** a permis à la MOA de vérifier les livrables :

- Sprint 1** : Conteneurs Docker opérationnels et topic Kafka fonctionnel ;
- Sprint 2** : Simulateur et topologies traitent correctement les données ;
- Sprint 3** : Le replay produit des résultats cohérents.

6.3 | Utilisation de Taiga (Kanban)

Pour gérer le flux de travail au quotidien, nous avons mis en place un **tableau Kanban** sur **Taiga**, organisé en 5 colonnes : *New*, *Ready*, *In Progress*, *Ready for Test* et *Done*.



Cet outil nous a permis de visualiser les User Stories, suivre leur état d'avancement et identifier les goulots d'étranglement dans le processus.

6.4 | Outils Agile Utilisés

Outil	Rôle	Référence cours
Taiga	Kanban board, suivi des User Stories	Ch.5 — Suivi du projet
GanttProject	Planification temporelle, chemin critique	Ch.7 — Planification
Git	Versioning du code source	Bonne pratique DevOps
Docker	Déploiement reproductible	Sprint 1

Table 6.2 – Outils Agile et leur correspondance avec le cours

Chapitre 7

Gestion des Risques

7.1 | Identification et Priorisation

Conformément au Chapitre 9 du cours (Les Risques), nous avons identifié, priorisé et défini des actions de prévention pour les risques de ce projet. La gestion des risques comprend 4 étapes : **identifier**, **prioriser**, **prévenir** et **suivre**.

7.1.1 Risques Techniques

#	Risque	Probabilité	Impact	Criticité
R1	Perte de données Kafka (le broker crash)	Moyenne	Élevé	Critique
R2	Latence du traitement (topologies trop lentes)	Faible	Moyen	Modéré
R3	Incompatibilité des versions Docker/Kafka/-Flink	Moyenne	Élevé	Critique
R4	Mémoire insuffisante (Docker consomme trop de RAM)	Moyenne	Moyen	Modéré
R5	Incohérence des données (Replay $\neq$ Temps réel)	Faible	Élevé	Modéré

Table 7.1 – Matrice des risques techniques

7.1.2 Risques de Gestion de Projet

#	Risque	Probabilité	Impact	Criticité
R6	Retard de livraison (complexité sous-estimée)	Élevée	Élevé	Critique
R7	Mauvaise répartition des tâches	Moyenne	Moyen	Modéré
R8	Manque de compétences (technologies nouvelles)	Élevée	Moyen	Modéré
R9	Communication défaillante MOA/MOE	Faible	Moyen	Faible

Table 7.2 – Matrice des risques de gestion de projet

7.2 | Plans de Mitigation

#	Action Préventive	Action Corrective
R1	Configuration de la rétention (7 jours) et réplication	Replay depuis le dernier offset connu
R2	Partitionnement des topics (3 partitions)	Augmenter les task slots Flink
R3	Figurer les versions dans docker-compose	Tester avec des versions antérieures
R6	Estimation par Planning Poker	Réduire le périmètre (prioriser)
R8	Auto-formation et prototypage en Sprint 0	Pair programming et support mutuel

Table 7.3 – Plans de mitigation des risques critiques

7.3 | Le Cône d’Incertitude

Référence au Chapitre 8 du cours (Estimation de charge). Au début du projet, l’incertitude liée à la technologie Kafka était très forte (**facteur $\times 4$**). Au fur et à mesure des Sprints, la maîtrise technique s’est améliorée :

- **Début du projet** : incertitude $\times 4$ (technologies inconnues Kafka/Flink) ;
- **Après Sprint 1** : incertitude $\times 2$ (l’infrastructure fonctionne) ;
- **Après Sprint 2** : incertitude $\times 1$ (les topologies sont validées).

7.4 | Suivi des Risques par Sprint

Risque	Sprint 1	Sprint 2	Sprint 3
R1 Perte données	Non traité	Rétention configurée	Validé
R2 Latence	Pas de problème	Acceptable	OK
R3 Versions	Résolu	–	–
R4 Mémoire	Docker lourd	Optimisé	OK
R6 Retard	Sprint 1 long	Rattrapage	Livré

Table 7.4 – Suivi de l’évolution des risques par sprint

Risque Technique Majeur

Les technologies Big Data/Streaming sont complexes à configurer. L’utilisation de `docker-compose` a permis d’encapsuler cette complexité et d’atténuer le risque de

problèmes d'environnement. En figeant les versions (`cp-kafka:7.3.0`, `flink:1.17`), nous avons éliminé le risque d'incompatibilité R3.

Conclusion et Perspectives

Bilan Technique

Ce projet a démontré avec succès la viabilité de l'**architecture Kappa** comme alternative à l'architecture Lambda pour le traitement événementiel en e-commerce. Les cinq composants développés — le simulateur, les topologies de filtrage, d'agrégation et de détection d'abandon, ainsi que le test de replay — fonctionnent de manière cohérente et valident les principes fondamentaux de cette architecture.

Le mécanisme de **replay**, pierre angulaire de Kappa, a prouvé sa fiabilité : les résultats obtenus en rejouant l'intégralité du journal d'événements sont **strictement identiques** à ceux calculés en temps réel, confirmant la parfaite cohérence du système.

Bilan Gestion de Projet

Sur le plan organisationnel, l'approche **Agile (Scrum + Kanban)** s'est révélée particulièrement adaptée à un projet comportant une forte incertitude technique. Le découpage en 4 sprints, la gestion rigoureuse des 9 risques identifiés et l'utilisation du Planning Poker pour l'estimation des charges ont permis de livrer le projet dans les délais avec une vélocité de 100%.

Leçons Apprises

- ▶ La **conteneurisation** (Docker) est indispensable pour encapsuler la complexité des architectures distribuées ;
- ▶ Le **Planning Poker** permet d'obtenir des estimations réalistes grâce au consensus d'équipe ;
- ▶ La **validation progressive** (recette par sprint) réduit considérablement les risques de non-conformité en fin de projet ;
- ▶ La **rétenion configurée par topic** permet d'adapter le coût de stockage au besoin métier réel.

Perspectives

Plusieurs axes d'évolution sont envisageables pour enrichir cette architecture :

1. **Machine Learning en temps réel** : intégrer des modèles prédictifs directement dans le flux Kappa pour prédire l'achat suivant d'un client pendant sa navigation ;

2. **Dashboard temps réel** : développer une interface graphique (Grafana, Kibana) pour visualiser les statistiques et alertes en direct ;
3. **Scaling horizontal** : déployer un cluster Kafka multi-brokers avec réplication pour garantir la haute disponibilité en production ;
4. **Tarification dynamique** : utiliser l'IA pour ajuster les prix en fonction du comportement client observé en temps réel.

Yazid TAHIRI ALAOUI, Souhayla ECH-CHARAY et Ouissal EL AMRANI
ENSAH – 2025 – 2026